DE 414: Practical 4 Report

Amy McDermott (26911264)

April 27, 2026

Introduction

This report documents the implementation of a Multi-layer Perceptron using Python. It explores the manual application of the forward pass, backpropagation, and loss calculation. The model's performance and ability to generalise are tested against the datasets `simple` and `two_blob`. By visualising the decision surfaces generated by the model, we can observe how the neural network's architectural choices - such as the number of hidden layers and neurons - impacts the network's capacity to handle various levels of data complexity.

Question 1: Single layer perceptron

In this section, a single-layer perceptron was implemented and trained on the `simple` dataset. This architecture consists of 2 input units, $P1 = 3$, and 1 output unit, $Q1 = 1$.

Loading of data

The first step of the practical involved loading the `simple` dataset and visualising its distribution. Using the `matplotlib` library, a scatter plot was generated with points colour-coded by their respective labels (red for class 0 and green for class 1).

Upon inspection of the resulting plot, the dataset consists of two clearly seperated clusters. Due to the fact that a straight line can be drawn to perfectly divide the two classes without any overlap, this dataset is classified as linearly seperable. On account of this, the single-layer perceptron (using a linear transformation and sigmoid activation) implemented in this section, is sufficient to achieve optimal classification performance.

Forward pass

The forward pass is the fundamental process by which the network transforms input data into a prediction. For the single layer implemented in this section, a linear layer output, z_i, and then the activation function output v_i is calculated. As demonstrated in Listing 1 below, z_i is computed by performing a matrix multiplication between the transpose of the weight matrix W and the input vector u_i (including a bias term). Following the linear transformation, the sigmoid activation function is applied to z_i to produce the final activation output, v_i.

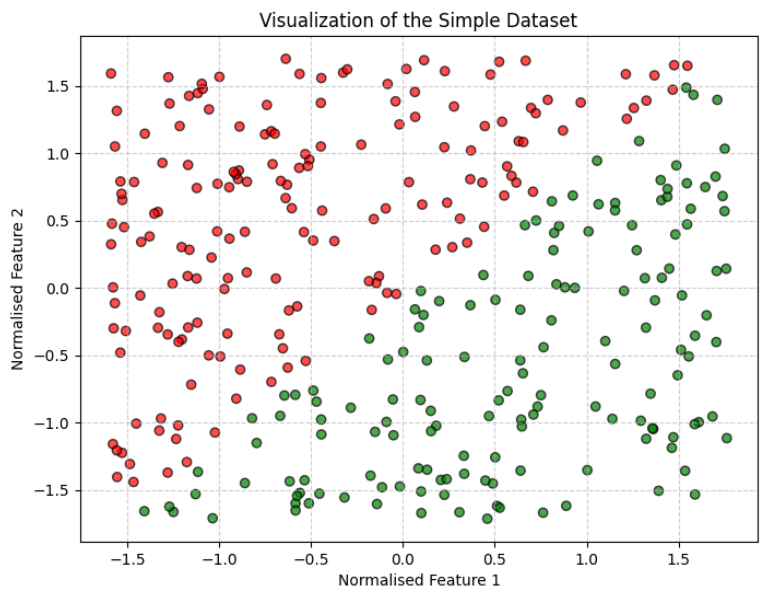

Figure 1: Scatter plot of the `simple` dataset, demonstrating linear separability.

```
z_i = W.T @ u_i  # Linear layer output (Transpose W to match dimensions for matrix multiplication)
v_i = sigmoid(z_i) # Activation function output
return z_i, v_i
```

Listing 1: Implementation of the forward pass for a single layer

Loss derivative

The optimisation of the network relies on the calculation of the gradient of the loss function (Binary Cross-Entropy) with respect to the model's predictions $\hat{y}$. This derivative serves as the initial error signal, Lambda, that is propagated backward through the network. As shown in the Listing 2 implementation below, the derivative of the binary cross-entropy loss is calculated using the following formula.

$$\frac{\partial L}{\partial \hat{y}} = \frac{y - \hat{y}}{\hat{y}(1 - \hat{y}) + \epsilon} \tag{1}$$

To ensure numerical stability and prevent a division-by-zero error when $\hat{y}$ is exactly 0 or 1, a small constant ϵ (1×10^{-7}) is added to the denominator.

```
eps = 1e-7
dL_dy_hat = None
dL_dy_hat = (y - y_hat) / (y_hat * (1 - y_hat) + eps)  # Add eps to avoid division by zero
return dL_dy_hat
```

Listing 2: Implementation of the derivative of the binary cross-entropy

Backward pass

The backward pass is the mechanism by which the network propagates the error signal from the output layer back toward the input, enabling the update of the weights via gradient descent. This process begins by computing the gradient of the loss with respect to the linear output z_i, where Λ is the error signal.

$$\delta_i = \Lambda \odot v_i(1 - v_i) \tag{2}$$

This result is subsequently used to determine the gradient of the loss with respect to the weights, $\frac{\partial J}{\partial W_i}$.

$$\frac{\partial J}{\partial W_i} = u_i \delta_i \tag{3}$$

A new Λ is then calculated to be passed to the preceeding layer. However this is only necessary for deeper architectures, where more than one layer makes up the network. When this updated Λ is computed, the first row of the weight matrix W_i is excluded since this row exists to incorporate a bias term and does not receive an error signal from previous layers. Listing 3 shows the necessary code to execute the backpass functionality.

```
delta = Lambda * v_i * (1 - v_i)  # Backpropagation factor for layer i (Q_i,1)
dJ_dWi = u_i @ delta.T  # Gradients of weights for layer i
Lambda = (W_i[1:, :] @ delta).flatten() # Backpropagation factor for previous layer (Q_i,1)
return dJ_dWi,Lambda
```

Listing 3: Implementation of the backward pass

Model training

Following the implementation of the forward and backward passes, the model was trained for 100 epochs using the default hyperparameters (learning rate of 0.1 and batch size of 1). Based on the training output, the model performance evolved as follows:

1. Epoch 0: test loss of -0.6572
2. Epoch 99: test loss of -0.2042

Table 1 illustrates model output over the last 10 epochs of training.

In this specific implementation, a value closer to zero indicates higher likelihood and therefore a better model fit. It is evident that the significant reduction in loss over model training demonstrates that the single-layer perceptron successfully converged.

Model decision surface

After training the model, the decision surface was visualised. The resulting plots, shown in Figure 2 displays a clear, linear seperation between the two classes, which confirms that the single-layer perceptron has successfully learned to partition the data. It is important to note that the decision surface of a single-layer perceptron is linear. More complex decision boundaries can only be captured using multi-layer architectures. Considering the distribution of the `simple` dataset, as seen in Figure 1, a linear decision surface, and hence a single-layer perceptron, is sufficient to achieve optimal classification performance.

Table 1: Training and test loss for the final 10 epochs of the single-layer perceptron on the `simple` dataset.

Epoch	**Training Loss**	**Test Loss**
90	-0.1618	-0.2166
91	-0.1545	-0.1885
92	-0.1436	-0.2212
93	-0.1369	-0.2294
94	-0.1278	-0.2491
95	-0.1302	-0.1579
96	-0.1336	-0.2296
97	-0.1310	-0.1639
98	-0.1227	-0.1945
99	-0.1153	-0.2042

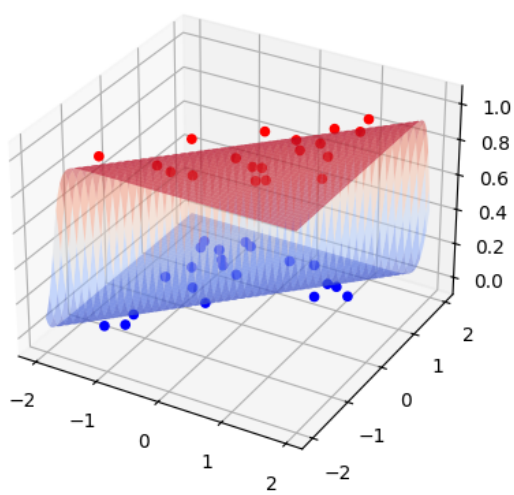

Figure 2: Decision surface of the `simple` data